

Guide Détaillé — Nour

Firmware Zephyr RTOS + Agent 3 (Générateur) + Frontend React + Monitoring

Ton périmètre en un coup d'œil

Élément	Détail
Firmware	Zephyr RTOS sur ESP32 (capteur DHT22 + publication MQTT)
Agent IA	Agent 3 — Générateur (templates Jinja2 pour Dockerfiles/configs)
Frontend	React + TypeScript (formulaire GitHub, suivi pipeline, chat RAG, dashboards Grafana en iframe)
Monitoring	Connexion MQTT → InfluxDB

Semaine 1 — Environnement Zephyr + premier projet "Blinky"

■ **Zephyr RTOS** : un système d'exploitation temps réel (RTOS) open source conçu pour les microcontrôleurs, utilisé par de nombreuses entreprises industrielles (contrairement à Arduino, plus simple mais moins "professionnel"). ■ **RTOS (Real-Time Operating System)** : un système d'exploitation conçu pour exécuter des tâches avec des contraintes de temps strictes (contrairement à un OS classique comme Windows).

Installation

```
bash
pip install west
west init ~/zephyrproject
cd ~/zephyrproject
west update
west zephyr-export
pip install -r ~/zephyrproject/zephyr/scripts/requirements.txt
```

Premier projet : Blinky (adapté de l'exemple officiel, ne pars jamais de zéro)

```
bash
```

```
cd ~/zephyrproject/zephyr
west build -b esp32_devkitc_wroom/esp32/procu samples/basic/blinky
west flash
```

 `west build -b ...` : compile le projet pour la carte cible précisée (ici l'ESP32). `west flash` : transfère le binaire compilé sur la carte via USB.

Livable : la LED de l'ESP32 clignote, pilotée par du code Zephyr (pas Arduino).

Maquette React (en parallèle si tu as le temps cette semaine)

```
bash
npm create vite@latest frontend -- --template react-ts
cd frontend
npm install
npm run dev
```

Livable : une page React vide qui tourne en local (`localhost:5173`).

Semaine 2 — Zephyr + MQTT + vues React

Adapter l'exemple MQTT officiel de Zephyr

```
bash
west build -b esp32_devkitc_wroom/esp32/procu samples/net/mqtt_publisher
```

Modifie le fichier `prj.conf` de cet exemple pour pointer vers ton broker Mosquitto (celui de Sahar) :

```
CONFIG_MQTT_LIB=y
CONFIG_NET_CONFIG_PEER_IPV4_ADDR="ADRESSE_IP_DU_BROKER"
```

Livable : l'ESP32 publie un message MQTT de test (même sans capteur réel encore) vers le broker de Sahar.

Vues React de base

```
tsx
```

```
// src/App.tsx
import { useState } from "react";

function App() {
  const [urlGithub, setUrlGithub] = useState("");

  const soumettre = () => {
    console.log("Soumission :", urlGithub);
    // Appel API à venir semaine 4
  };

  return (
    <div>
      <h1>Plateforme IoT DevOps AI</h1>
      <input
        type="text"
        placeholder="URL du dépôt GitHub"
        value={urlGithub}
        onChange={(e) => setUrlGithub(e.target.value)}
      />
      <button onClick={soumettre}>Analyser</button>
    </div>
  );
}

export default App;
```

Livrable : un formulaire fonctionnel (même sans backend connecté).

Semaine 3 — Capteur réel + Vues React avancées

Lire un capteur DHT22 sous Zephyr

c

```
// main.c (extrait)
#include <zephyr/kernel.h>
#include <zephyr/drivers/sensor.h>

void main(void) {
    const struct device *dht = DEVICE_DT_GET_ONE(aosong_dht);
    struct sensor_value temp, hum;

    while (1) {
        sensor_sample_fetch(dht);
        sensor_channel_get(dht, SENSOR_CHAN_AMBIENT_TEMP, &temp);
        sensor_channel_get(dht, SENSOR_CHAN_HUMIDITY, &hum);
        printk("Température: %d.%06d, Humidité: %d.%06d\n",
               temp.val1, temp.val2, hum.val1, hum.val2);
        k_sleep(K_SECONDS(5));
    }
}
```

■ **Device Tree** : un fichier qui décrit le matériel physique connecté à la carte (quel capteur, sur quelle broche) — Zephyr l'utilise pour savoir comment piloter le DHT22 sans que tu écrives le pilote toi-même.

Livable : les valeurs de température/humidité s'affichent dans le terminal série, puis sont publiées en MQTT (combine avec le code de la semaine 2).

Vues React : suivi de pipeline + logs

Ajoute une zone d'affichage qui simule (pour l'instant) les étapes du pipeline : "Analyse... Architecture... Génération... Build... Déploiement." — tu connecteras les vraies données en semaine 4.

Semaine 4 — Connexion MQTT → InfluxDB

(Identique à l'ancienne version : script Python `paho-mqtt` + `influxdb-client` qui écrit les mesures reçues dans InfluxDB — voir le code déjà fourni précédemment si besoin, la logique ne change pas.)

Semaine 5 — Agent 3 (Générateur)

Contrat d'interface reçu d'Ameni (Agent 2) :

```
json
```

```
{
  "strategie_build": "docker_west_build",
  "ota_active": false,
  "monitoring": true,
  "broker_mqtt": "mosquitto",
  "justification": "Projet Zephyr avec MQTT, build isolé nécessaire"
}
```

Crée ce fichier toi-même en `mock_agent2_output.json` et commence sans attendre Ameni.

 **Jinja2** : un moteur de templates Python — au lieu de faire écrire le fichier complet par le LLM à chaque fois, on prépare un modèle avec des "trous" que le code remplit automatiquement, ce qui est plus fiable et plus rapide.

```
python

from jinja2 import Template
import json

template_dockerfile = Template("""
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y git cmake ninja-build python3-pip
RUN pip3 install west
{% if strategie_build == "docker_west_build" %}
ENTRYPOINT ["west", "build"]
{% endif %}
""")

def generer_fichiers(decision: dict) -> dict:
    dockerfile_genere = template_dockerfile.render(**decision)
    return {"dockerfile": dockerfile_genere}

with open("mock_agent2_output.json") as f:
    decision_mock = json.load(f)

fichiers = generer_fichiers(decision_mock)
with open("generated/Dockerfile", "w") as f:
    f.write(fichiers["dockerfile"])
```

Version avec LLM pour des cas plus complexes (variables selon OTA, monitoring...) :

```
python
```

```

import anthropic

client = anthropic.Anthropic(api_key="TA_CLE_API")

def generer_fichiers_llm(decision: dict) -> dict:
    prompt = f"""
    À partir de cette décision d'architecture : {json.dumps(decision)}
    Génère un Dockerfile adapté pour un Build Runner Zephyr.
    Réponds UNIQUEMENT en JSON avec le champ "dockerfile".
    """

    response = client.messages.create(
        model="claude-sonnet-4-5", max_tokens=1000,
        messages=[{"role": "user", "content": prompt}]
    )
    return json.loads(response.content[0].text)

```

Livable : l'Agent 3 génère un Dockerfile valide, testé avec `docker build`.

Semaine 6 — Firmware complet + Dashboards Grafana + intégration React

1. Finalise le firmware Zephyr (capteur + MQTT stable, testé sur plusieurs heures pour vérifier la fiabilité).
2. Termine les dashboards Grafana (en coordination avec Ameni sur l'infra).
3. Intègre les graphiques Grafana dans React via iframe :

```

tsx

<iframe
  src="http://localhost:3000/d/mon-dashboard?orgId=1&refresh=5s&theme=light&kiosk"
  width="100%"
  height="400"
  title="Dashboard capteurs"
/>

```

4. Connecte le chat RAG : un champ de saisie qui appelle l'endpoint `/assistant` du backend (Sahar) et affiche la réponse.

Livable : l'interface React affiche le dashboard en temps réel et permet de discuter avec l'Agent 4.

Semaine 7 — Intégration finale

Liaison complète de l'UI React avec les API du backend (Sahar/Ameni) : affichage du pipeline en temps réel, chat RAG synchrone, tests de bout en bout.

Ta soupape de sécurité

1. Sacrifie en premier : le style visuel avancé de React (Tailwind, animations) — une interface simple et fonctionnelle suffit.
 2. Sacrifie ensuite : le firmware Zephyr avancé — si le temps presse, reviens temporairement au simulateur Python en MQTT pour garantir une démo fonctionnelle, et documente Zephyr comme travail en cours.
 3. Ne sacrifie jamais : la publication de `mock_agent2_output.json` que tu utilises — et préviens immédiatement Sahar/Ameni si tu as besoin d'un vrai changement de format.
-

Combine ce document avec le cahier des charges fusionné pour la vue d'ensemble.